揮発性サイバー防御（Volatile Cyber Defense）

攻撃検知をトリガーとした自壊・証拠保全・自律回復による新たな防御パラダイム

著者：山下 建

キーワード：Volatile Cyber Defense, Intentional Crash, Forensics-First Termination, Self-Destructing System, Intrusion Response, Elixir/OTP, Kubernetes, Defense in Depth

---

Abstract

現代のセキュリティ設計は「攻撃を防ぐ」ことを前提とし、侵入後の汚染状態への対処を事後のフォレンジック調査に委ねている。しかし攻撃者が第一防御層を突破した場合、汚染された実行状態はシステムに残留し、横展開・長期潜伏・証拠隠滅の足場となる。

この問題に対する新たな防御パラダイムとして、揮発性サイバー防御（Volatile Cyber Defense、VCD）を提案する。VCDは「攻撃を検知したらプロセスが即座に証拠を保全し自壊、クリーンな状態で回復する」という設計原則に基づく。従来のMoving Target Defense・Phoenix Servers・Intrusion Toleranceとは異なり、VCDは検知をトリガーとした即時自壊・フォレンジック優先終了・自律回復を同時に設計原則とする点において先行研究に類例を持たない。

本論文はElixir/OTPおよびKubernetesを用いたプロセスレベル・コンテナレベルの多段揮発性実装を示し、VCDの技術的実現可能性を実証する。

1. はじめに

現代のサイバーセキュリティは、多層防御（Defense in Depth）やゼロトラスト（Zero Trust）に代表されるように、攻撃を検知・遮断し、システムの継続稼働を維持することを基本方針としている。しかしこれらの設計は、攻撃者が第一防御層を突破した後の状態を十分に考慮していない。

攻撃者が侵入に成功した場合、汚染された実行状態はシステムのメモリ・セッション・変数に残留し続ける。攻撃者はこの状態を足場として、横展開・長期潜伏・証拠隠滅を行う。従来のフォレンジック手法はこうした事象を事後に調査・復元するものであり、侵入中の汚染状態の拡大を防ぐ手段を持たない。

本研究はこの問題を「汚染状態の残留問題」として定義し、設計レベルでの解決を試みる。提案するのは、攻撃を検知したコンポーネントが即座に証拠を保全して自壊し、クリーンな状態で自律回復するという新たな防御パラダイム——揮発性サイバー防御（Volatile Cyber Defense、VCD）である。

VCDはErlang/OTPの「Let it crash」哲学とKubernetesの自律回復機構から着想を得て、「壊れることを前提とした設計」をセキュリティの文脈に応用する。自壊は障害ではなく設計された防御行動であり、攻撃者が汚染状態を利用する機会を構造的に排除する。

クラウドネイティブ時代の到来により、VCDの実現基盤は整いつつある。コンテナ化とKubernetesに代表されるオーケストレーション技術は、プロセスやコンテナの廃棄・再生成をインフラレベルで自動化した。物理サーバー時代には「壊れることを前提とした設計」はコストとリスクを伴う選択肢であったが、今日では軽量なコンテナの破棄と再起動は日常的な運用の一部となっている。VCDはこの基盤の上に、セキュリティという新たな目的を与えるものである。

本論文の貢献は以下の通りである。

- 汚染状態の残留問題を定式化し、揮発性サイバー防御（VCD）として体系化する
- MTD・Phoenix Servers・Intrusion Toleranceとの比較によりVCDの独自性を明確にする
- Elixir/OTPおよびKubernetesを用いた多段揮発性の実装によりVCDの技術的実現可能性を示す

本論文の構成は以下の通りである。2章で関連研究を整理し、3章でVCDのパラダイムとアーキテクチャを詳述する。4章で実装を示し、5章で考察を行い、6章でまとめる。

2. 関連研究

2.1 Moving Target Defense（MTD）

Moving Target Defense（MTD）は、システムの構成・ネットワークトポロジー・IPアドレス等を継続的に変化させることで攻撃面を動的に変化させ、攻撃者の偵察・侵入を困難にするセキュリティ戦略である[1]。MTDの主目的は攻撃者に標的を見つけさせないことにあり、予防的（Proactive）なアプローチである。

VCDはMTDと「静的な状態の維持を前提としない」という点で共通するが、MTDが攻撃前の予防を目的とするのに対し、VCDは攻撃検知後の即時対応を設計原則とする。またMTDはフォレンジック証拠の保全を目的としない点でも異なる。両者は対立するものではなく、MTDで侵入を困難にしつつ、突破された場合にVCDが機能するという多段防御として組み合わせが可能である。

2.2 Phoenix Servers

Phoenix Serversは、サーバーを定期的に破棄・再構築することで設定ドリフトを排除し、常に既知のクリーンな状態を維持するインフラ設計パターンである（Fowler, 2012）[2]。Immutable Infrastructureの基盤概念でもあり、Kubernetesのコンテナ設計思想とも親和性が高い。

VCDとPhoenix Serversは「破棄してクリーンな状態に戻る」という思想を共有するが、Phoenix Serversがリリース・運用管理を目的としたスケジュールベースの再生成であるのに対し、VCDは攻撃検知をトリガーとした即時自壊とフォレンジック証拠の保全を設計原則とする点が本質的に異なる。

2.3 Intrusion Tolerance

Intrusion Toleranceは、すべての侵入を防ぐことを諦め、侵入されてもシステムのセキュリティ障害につながらないようにするメカニズムを設計するアプローチである[3]。冗長性・多様性・Byzantine Fault Toleranceを基盤とし、侵入下での継続稼働を目標とする。

VCDとIntrusion Toleranceはともに「侵入されることを前提とした設計」である点で従来の防御モデルと異なる。しかしIntrusion Toleranceが「侵入されても動き続ける」ことを目標とするのに対し、VCDは「侵入を検知したら意図的に止まり、クリーンに再生する」ことを目標とする。目標が継続か消去かで設計思想が対照的である。

2.4 Digital Forensics and Incident Response（DFIR）

Digital Forensics and Incident Response（DFIR）は、インシデント発生後にデジタル証拠を収集・分析し、攻撃の全容を解明する手法である[4]。近年はProactive Forensicsとして、インシデント発生に備えた事前準備を重視する方向性も提唱されている。

VCDはDFIRの「証拠保全」という目的を共有するが、DFIRが事後の調査・収集を前提とするのに対し、VCDはシステム自身が自壊前に証拠を自律的に送出する点が異なる。攻撃者が証拠を隠滅する前にシステムが証拠を保全するという点で、VCDはDFIRを補完する設計である。

2.5 VCDの独自性

以上の比較から、VCDの独自性は以下の3点の組み合わせにある。

- 検知をトリガーとした即時自壊（Reactive Intentional Crash）
- 自壊前のフォレンジック優先送出（Forensics-First Termination）
- 自律回復とクリーンな状態への復帰

この3つを同時に設計原則とする概念は上記の先行研究には存在しない。VCDは既存概念と対立するものではなく、これらと組み合わせることで多層的な防御を実現するパラダイムとして位置づけられる。

3. 揮発性サイバー防御（VCD）

3.1 揮発性（Volatility）の定義

VCDにおける「揮発性」とは、攻撃や異常を検知したプロセス・コンテナが、自律的に実行状態を消去して再起動する性質を指す。化学における揮発（蒸発して痕跡を残さない）と同様に、汚染された実行状態が「蒸発」するという比喩に由来する。

従来のセキュリティ設計が「防御して継続する」ことを前提とするのに対し、VCDは「検知したら意図的に死んで、クリーンに蘇る」ことを設計の中核に置く。この発想はErlang/OTPの「Let it crash」哲学——障害を例外ではなく通常の状態として扱い、回復の責任を上位の仕組みに委譲する——をセキュリティの文脈に転用したものである[5]。

揮発性はシステムの脆弱性ではなく、設計された防御特性である。攻撃者が汚染状態を利用して横展開・長期潜伏を試みようとしても、その状態が蒸発することで機会を構造的に失う。

3.2 自壊（Intentional Crash）

VCDにおいて自壊は障害ではなく、設計された防御行動である。攻撃を検知したコンポーネントが自律的に終了することで、以下を実現する。

- 攻撃者が汚染したメモリ・セッション・変数を一切引き継がない
- 汚染状態の横展開を構造的に遮断する
- セッションハイジャック後の長期潜伏を防ぐ

自壊の実装においては、クリティカルパスの最短化が重要な設計原則となる。「攻撃者より先に自壊する」ためには、検知から自壊完了までのレイテンシを最小にする必要があるからである。VCDでは自壊直前の処理をローカルへのファイル書き込みのみとし、データ転送・永続化・外部連携などの重い処理は上位レイヤーまたは別コンポーネントに委譲する。これにより自壊のクリティカルパスは以下の2ステップに収まる。

```
異常検知 → ファイル書き込み → exit
```

委譲先のコンポーネントはファイルの変更を検知して非同期に後続処理を引き継ぐ。自壊したコンポーネントが終了している間も後続処理が証拠送付を完了できるため、自壊の速度と証拠保全の確実性を両立できる。

3.3 フォレンジック優先終了（Forensics-First Termination）

VCDの自壊コンポーネントは、終了前に必ずフォレンジック情報を外部に送出する。この設計原則を「フォレンジック優先終了」と呼ぶ。

従来のフォレンジックが「事後の調査・復元」であるのに対し、VCDはシステム自身が自壊前に証拠を自律的に保全する。攻撃者が証拠を隠滅する機会を得る前に、以下の情報がコンポーネント外の永続ストレージに記録される。

- 接続元IPアドレス・セッション情報
- 検知した異常の種別と内容（検知直前のリクエスト・クエリ）
- タイムスタンプ（高精度）
- コンポーネントのメモリ状態・スタックトレース・メッセージキューの状態

特にElixir/OTPのようなランタイムでは、プロセスが保持するメモリ・リダクション数・スケジューラ情報といったVM内部状態もフォレンジック情報として取得できる。これにより、攻撃の侵入経路や影響範囲の再現性が高まる。

フォレンジック優先終了において重要なのは、証拠送出とコンポーネント終了の順序保証である。送出が完了する前にコンポーネントが死ぬリスクに対しては、3.2で述べたクリティカルパスの最短化——ローカルファイルへの書き込みのみを自壊前に行い、データ転送は別コンポーネントに委譲する——によって対処する。

3.4 責務の分離

VCDは「自壊する責務」と「証拠を送る責務」を明確に分離する。この分離により、自壊したコンポーネントが終了しても証拠送付を完了できる構造を実現する。

コンポーネント	責務
自壊コンポーネント	検知・ファイル書き込み・自壊のみを担う
後続コンポーネント	ファイル監視・データ転送・外部連携のみを担う

この分離には重要な原則が伴う。後続コンポーネントから自壊コンポーネントへの逆方向の通信は禁止する。外部からの指示・フィードバックを受け付けないことで、後続コンポーネントや外部ストレージが侵害された場合でも自壊コンポーネントの挙動に影響しない。

責務の分離はクリティカルパスの最短化とも密接に関係する。自壊コンポーネントが「ファイル書き込みと自壊」だけに専念できるのは、後続の重い処理を別コンポーネントに委譲しているからである。

3.5 並列検査（Parallel Validation）

複数の入力フィールドが存在する場合、逐次的な検査では検査時間が入力数に比例して増加する。VCDでは全入力フィールドを並列に検査し、いずれか1つでも攻撃を検知した時点で即座に自壊フローへ移行する設計を採用する。

```
入力A ─┐
入力B ─┼─ 並列検査 ─ いずれかが検知 ─ 自壊フローへ
入力C ─┘
```

この設計には2つの効果がある。第一に、入力フィールドの数に関わらず検査時間を1フィールド分に近づけることで、クリティカルパスの最短化に貢献する。第二に、最初に検知したプロセスが即座に自壊フローを開始することで、他の検査結果を待たずに対応を開始できる。

並列検査において重要なのはタイムアウトの設計である。いずれかの検査が応答しない場合にシステム全体が停止しないよう、各検査には上限時間を設ける。ただしタイムアウト時の扱いは慎重な設計が必要であり、巨大なデータに攻撃を埋め込むことで検査をタイムアウトさせる回避手法への対処も考慮しなければならない。タイムアウト時の判断方針はシステムの要件に応じて設計する必要がある。

3.6 多段揮発性レイヤー

VCDはプロセス単一レイヤーのみでも実現可能であり、攻撃を検知したプロセスが自壊・回復するだけでも揮発性防御として機能する。さらに粒度の異なる複数のレイヤーを組み合わせることで、多段的な揮発性防御を実現できる。各レイヤーは下位レイヤーが回復できない障害を上位レイヤーが引き受けるという階層構造を持つ。

レイヤー	担当	回復の単位	応答速度
L1: プロセス	OTP Supervisor	個別プロセス	ミリ秒〜秒

L2: コンテナ	Kubernetes Pod	コンテナ単位	秒〜数十秒
L3: ノード	Kubernetes Cluster	ノード単位	数分

L1では、攻撃を検知したプロセスが自壊し、Supervisorが即座にクリーンなプロセスを再生成する。軽微な攻撃はL1で完結し、コンテナ全体への影響を与えない。

L2では、L1での回復が困難な高深刻度の攻撃に対してコンテナ全体を破棄・再生成する。L1の自壊がトリガーとなりL2へescalateするかどうかは、攻撃の深刻度に応じて判断する（3.8節参照）。

L3は、Kubernetesがノード障害を検知した場合に自律的に処理するものであり、VCDが明示的に制御する対象ではない。L1・L2の多段揮発性が機能した結果、L3への到達を防ぐことがVCDの設計目標である。

この階層設計により「軽い攻撃はL1が吸収し、重い攻撃はL2が処理する」という自然な役割分担が生まれる。なお、3.7節以降はL1・L2の多段構成を前提として説明する。

3.7 Graceful Volatile Shutdown

多段揮発性レイヤーを構成する場合、上位レイヤーの自壊時には下位レイヤーで処理中のリクエストへの影響を考慮する必要がある。L1の自壊は該当プロセスのみに影響を与えるが、L2の自壊はその配下で処理中の全プロセスに影響を及ぼすためである。

VCDはこの問題に対してGraceful Volatile Shutdownという4段階のシャットダウンシーケンスを定義する。

```
① 攻撃を受けたL1プロセスが即座に自壊（証拠保全 → exit）
② L2コンテナが新規リクエストの受付を停止
③ 処理中の他プロセスが既存リクエストを完了
④ 全プロセスの完了後にL2コンテナが終了・再生成
```

②の新規受付停止は、インフラ側の仕組みを活用して実現する。アプリケーションがヘルスチェックエンドポイントの応答を変化させることで、オーケストレーション基盤がトラフィックの切り離しを自律的に行う。アプリケーションがインフラ側を直接操作する設計は採用しない。

③の処理継続フェーズでは、同一L2コンテナ内の他プロセスが処理しているリクエストも攻撃に関連している可能性を否定できない。このリスク許容度はシステムの要件に応じて以下のモードで設定する。

モード	動作	想定ユースケース
strict	攻撃検知と同時に全プロセスを即時破棄	高セキュリティ要件
graceful	処理中リクエストの完了を待って終了	可用性とセキュリティのバランス重視
timeout	最大N秒待って強制終了	タイムアウト値でリスク許容度を調整

なおtimeoutの上限値はインフラ側のタイムアウト設定より短く設定する必要がある。これにより、アプリケーションが自律的にシャットダウンを完了できなかった場合でも、インフラ側が強制的にL2コンテナを終了させる安全網が機能する。

3.8 深刻度による分岐（Severity-based Branching）

攻撃を検知した場合に、常にL2の自壊まで進む必要はない。攻撃の深刻度に応じてL1のみで完結するかL2にescalateするかを分岐することで、不必要なL2再起動を避けながら適切な対応を実現する。

深刻度	自壊レベル	判断基準の例
低（low）	L1のみ	単発のXSS・SQLi検知
高（high）	L1 → L2	短時間に同一IPから複数回・破壊的なSQLの検知

深刻度の判断基準はシステムの要件に応じて設計する。本論文の実装では、同一IPからの攻撃回数を一定期間内にカウントし、閾値を超えた場合にhighと判断する方式を採用した（詳細は4章参照）。

深刻度判定において重要なのは、判定ロジック自体がクリティカルパスに影響を与えないことである。VCDでは判定ロジックを軽量に保ち、詳細な分析は別コンポーネントに委譲する設計を推奨する。

3.9 データフロー設計

VCDにおけるデータの流れは一方向性を原則とする。L1コンポーネントから外部へ証拠を送出する方向のみを許可し、外部からL1コンポーネントへの逆方向の通信は禁止する。この原則により、外部コンポーネントが侵害された場合でもL1コンポーネントの挙動に影響しない。

```
L1コンポーネント → ファイル書き込み（自壊前）
                        ↓
後続コンポーネント → データ転送（検知後・非同期）
                        ↓
外部ストレージ    → 永続化・ブロックリスト連携・ネットワーク層への反映
```

コンポーネント	役割	通信方向
L1コンポーネント	検知・証拠書き込み・自壊	→ ファイルのみ
後続コンポーネント	ファイル監視・データ転送	ファイル → 外部ストレージ
外部ストレージ	証拠の永続化・ブロックリスト管理	→ ネットワーク層
ネットワーク層	接続元IPのブロック	なし（受信のみ）

ネットワーク層でのブロックにより、攻撃者の接続はL1コンポーネントに到達する前に遮断される。L1層でのブロックと組み合わせることで、多段的なブロック機構を実現する。

外部ストレージへのデータ転送は非同期で行われるため、転送が完了する前にL1コンポーネントが再起動した場合でも証拠が失われない。ファイルへの書き込みはL1コンポーネントが自壊する前に同期的に完了しており、後続コンポーネントが後から読み取ることができるためである。

4. 実装

4.1 実装方針

本章では、3章で定義したVCDのパラダイムとアーキテクチャが技術的に実現可能であることを示す。実装は3つのフェーズに分けて段階的に進めた。各フェーズは前フェーズの概念実証を引き継ぎ、より完全なVCDの実現を目指す。

フェーズ	言語・技術	主な実証内容
Phase1	Python / Flask	VCDの基本的な自壊・回復・ブロックの動作確認
Phase2	Elixir / OTP	OTPによるプロセスレベル揮発性の実現
Phase3	Elixir + Kubernetes	多段揮発性・Graceful Volatile Shutdownの実現

各フェーズの実装はすべて公開リポジトリで参照できる。本実装はプロダクションレベルの品質を目的とするものではなく、VCDの概念が技術的に成立することを示す概念実証（Proof of Concept）である。

4.2 Phase1：概念実証（Python / Flask）

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase1/python

Phase1ではPythonおよびFlaskを用いて、VCDの基本的な動作を最小構成で実装した。目的は「検知→証拠保全→自壊→回復→ブロック」の5ステップが一連の流れとして動作することを確認することである。

実装した機能は以下の通りである。

- XSS・SQLiパターンの正規表現による検知
- 検知直前の情報をローカルファイルへ追記（append-only）
- os._exit(1)による即時自壊
- supervisordによる自動再起動
- 起動時のブロックリスト読み込みと同一IPへの403応答

Pythonを選択した理由は、概念実証の段階では実装の簡潔さを優先したためである。OTPのようなプロセス管理機構を持たないPythonでVCDの核心部分が実現できることは、VCDが特定の言語や実行環境に依存しない汎用的なパラダイムであることを示す。

4.3 Phase2：Elixir / OTP

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase2/elixir

Phase2ではElixirおよびOTPを用いて、VCDの設計思想と最も親和性の高い実装を行った。Phase1との最大の違いは、プロセス管理・状態管理・並列処理がランタイムレベルで支援される点である。

実装した機能は以下の通りである。

- Plug + Cowboyによるリクエスト受付
- Task.async_streamによる全入力フィールドの並列検査
- GenServer + ETSによるブロックリスト管理（プロセス再起動後も状態が維持される）
- exit(:attack_detected)による即時自壊

- OTP Supervisorによる自動再起動
- 自壊前のVM内部状態（メモリ・リダクション数・スタックトレース）を含むフォレンジック情報の書き出し

Phase2ではPhoenixを使用せずPlug単体で実装した。VCDのPoCに必要な機能はHTTPリクエストの受付と検知ロジックのみであり、フルスタックフレームワークは過剰であると判断したためである。

4.4 Phase3：Elixir + Kubernetes

https://github.com/nanikore7250/VolatileCyberDefense/tree/phase3/k8s

Phase3ではPhase2の実装にKubernetesを統合し、L1・L2の多段揮発性とGraceful Volatile Shutdownを実現した。

Phase3で追加・拡張した機能は以下の通りである。

- ShutdownState GenServerによるL1・L2分岐の管理
- Readiness Probeエンドポイントによる Kubernetesへの自壊通知
- 深刻度判定（同一IPからの攻撃回数が閾値を超えた場合にL2へescalate）
- Graceful Volatile Shutdownの3モード（strict・graceful・timeout）
- emptyDirボリューム経由での別コンテナとのフォレンジックファイル共有
- Sidecarコンテナによるinotifyを用いたファイル監視とデータ転送
- Sidecarによる定期的なNetworkPolicyの更新（ネットワーク層でのIPブロック）

L2の自壊通知はReadiness Probeを活用して実現した。ShutdownStateがL2自壊を決定した時点でReadiness Probeエンドポイントが503を返すよう切り替わり、KubernetesがPodをServiceから自動的に切り離す。アプリケーションがKubernetes APIを直接操作する設計は採用しなかった。

フォレンジック情報の転送とNetworkPolicyの更新はSidecarコンテナが担う。アプリケーションコンテナはファイルへの書き込みのみを行い、SidecarがinotifyでファイルのSidecarがinotifyでファイルの変更を検知して後続処理を引き継ぐ。Sidecarからアプリケーションコンテナへのフィードバックはなく KubernetesのNetworkPolicyで一方向性を保証している。

5. 考察

5.1 誤検知（False Positive）問題

VCDの自壊はリクエストの処理を即座に中断するため、誤検知が発生した場合は正常なユーザーへの影響が大きい。特にL2の自壊が誤検知によってトリガーされた場合、コンテナ全体が再起動するためサービス断が発生する。

本実装では正規表現によるXSSおよびSQLiの検知を採用しているが、サイバー攻撃の種類はこれらに限られない。コマンドインジェクション・パストラバーサル・バッファオーバーフロー・ゼロデイ攻撃など、既知のパターンに収まらない攻撃は検知できない。また「何をもって攻撃とみなすか」という検知の定義自体が未解決の問題であり、VCDの有効性は検知ロジックの精度に強く依存する。本論文では検知ロジックの詳細には踏み込まず、この点を今後の研究課題として位置づける。

誤検知の影響を最小化する設計として、以下が考えられる。

- 深刻度判定の精緻化（単発の検知ではL2にescalateしない）
- 機械学習による意味的な検知への移行
- 誤検知が疑われる場合の人手による確認フローの導入

5.2 IPブロックの限界

本実装のブロック機構は接続元IPアドレスに基づく。この方式にはいくつかの限界がある。

第一に、NATやプロキシの背後に複数のユーザーが存在する場合、攻撃者と同じIPを持つ正常なユーザーが巻き込まれる可能性がある。第二に、攻撃者がIPアドレスを変更することで、ブロックを容易に回避できる。第三に、大量のIPアドレスからの分散攻撃（DDoS）に対しては、ブロックリストが際限なく膨張する。

これらの限界に対しては、IPアドレス以外の識別情報（セッションID・ユーザーエージェント・行動パターン）を組み合わせたブロック機構の設計が有効と考えられる。

5.3 自壊の原子性

VCDのクリティカルパスは「ファイル書き込み → exit」の2ステップであるが、ファイル書き込みの途中でプロセスが異常終了した場合、フォレンジック情報が不完全な状態でファイルに残るリスクがある。

この問題に対しては、ファイルへの書き込みをアトミックに行う設計が有効である。具体的には一時ファイルへの書き込み完了後にリネームする方式が考えられる。ファイルシステムにおけるリネーム操作はアトミックであるため、後続コンポーネントが読み取る時点では常に完全なエントリのみが存在することを保証できる。

5.4 Sidecar障害時の証拠保全

Sidecarコンテナが障害によって停止した場合、フォレンジック情報のデータ転送が行われない可能性がある。アプリケーションコンテナが書き込んだファイルはemptyDirボリュームに保存されているが、Podが再起動した場合はemptyDirの内容も失われる。

この問題に対しては以下の対策が考えられる。

- フォレンジックファイルをemptyDirではなく永続ボリュームに保存する
- SidecarをKubernetesのDaemonSetとして独立させ、アプリケーションPodのライフサイクルから切り離す
- Sidecarのヘルスチェックを強化し、障害時に即座に検知・再起動する

5.5 自壊機構を悪用したDoS攻撃

VCDの自壊機構は、攻撃者によって悪用される可能性がある。意図的に攻撃パターンを送信することでプロセスやコンテナを繰り返し自壊させ、サービス断を引き起こすDoS攻撃が考えられる。

この問題はVCDの設計上の根本的なトレードオフである。自壊の速度を上げるほどDoSへの脆弱性が増し、DoS対策を強化するほど自壊の判断が遅くなる。対策としては以下が考えられる。

- 一定時間内の自壊回数に上限を設け、上限を超えた場合は自壊を停止して警告を発する
- L2自壊の閾値を高く設定し、単純な攻撃パターンではL2まで到達しないようにする
- 上位レイヤーでのレートリミットとVCDを組み合わせる

5.6 今後の課題

本実装はアプリケーション組み込み型のVCDであり、既存システムへの導入にはアプリケーションの改修が必要となる。より広い普及を実現するためには、既存システムの前段に置くだけで導入できるproxy型VCDの実現が有効と考えられる。proxy型では、アプリケーションに手を加えることなくVCDの揮発性防御を適用できるため、言語・フレームワークを問わず導入可能となる。

またアプリケーション層に留まらず、nginx・Envoyなどのプロキシ層やファイアウォール・ネットワーク層へのVCDの適用も今後の研究課題である。各レイヤーでVCDが機能することを実証することで、VCDが特定の実装に依存しない汎用的な防御パラダイムであることを示せる。

検知精度の向上も重要な課題である。本実装では正規表現による検知を採用したが、機械学習を用いた意味的な検知への移行により誤検知率を低下させ、より高度な攻撃パターンへの対応が可能となる。

VCDの設計上の根本的な課題として、自壊させたくないプロセスの扱いが挙げられる。決済処理・長時間トランザクション・ステートフルなセッションなど、途中で自壊させることがビジネス上許容されないプロセスが存在する。proxy型への分離や機能の責務分割によって緩和できる可能性はあるが、「すべてのプロセスが自壊可能である」というVCDの前提と根本的に相容れない側面があり、今後の研究が必要である。

6. おわりに

本論文では、侵入後の汚染状態の残留問題に対する新たな防御パラダイムとして揮発性サイバー防御（Volatile Cyber Defense、VCD）を提案した。

VCDの核心は「検知したら意図的に死んで、クリーンに蘇る」という設計原則にある。攻撃を検知したコンポーネントが即座に証拠を保全して自壊し、クリーンな状態で自律回復することで、攻撃者が汚染状態を利用する機会を構造的に排除する。この思想はErlang/OTPの「Let it crash」哲学をセキュリティの文脈に転用したものであり、クラウドネイティブ時代のインフラが提供する廃棄・再生成の基盤の上に成立する。

本論文の貢献は以下の3点にまとめられる。

- 汚染状態の残留問題を定式化し、VCDとして体系化した
- MTD・Phoenix Servers・Intrusion Toleranceとの比較によりVCDの独自性を明確にした
- Elixir/OTPおよびKubernetesを用いた多段揮発性の実装によりVCDの技術的実現可能性を示した

VCDは既存のセキュリティパラダイムと対立するものではなく、「第一防御層突破後の防御」を担う層として既存の多層防御と組み合わせることで効果を発揮する。Defense in Depthの枠組みの中で、VCDは新たな一層として位置づけられる。

今後の課題としては、検知ロジックの精度向上・proxy型VCDの実現・プロキシ層やネットワーク層への適用拡大・自壊させたくないプロセスへの対応が挙げられる。これらの実証と解決を通じて、VCDが特定の実装に依存しない汎用的な防御パラダイムであることを示していく。

またVCDの発展的な方向性として、自壊の代わりに攻撃者を隔離された偽環境へ動的に誘導するハニーポット転換機能が考えられる。攻撃を検知した時点でクリーンな環境への自壊・回復を行う代わりに、攻撃者の行動を観察できる偽環境へシームレスに切り替えることで、フォレンジック情報の収集をさらに深化させることができる。この機能はVCDの「証拠保全」という思想の延長線上にあり、OSSとしての発展において有望な方向性の一つである。

参考文献

[1] Jajodia, S., Ghosh, A. K., Swarup, V., Wang, C., & Wang, X. S. (Eds.). (2011). Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats. Springer.

https://link.springer.com/book/10.1007/978-1-4614-0977-9

[2] Fowler, M. (2012). PhoenixServer. martinfowler.com.

https://martinfowler.com/bliki/PhoenixServer.html

[3] Veríssimo, P. E., Neves, N. F., & Correia, M. P. (2003). Intrusion-Tolerant Architectures: Concepts and Design. In Architecting Dependable Systems, LNCS vol. 2677, pp. 3–36. Springer.

https://link.springer.com/chapter/10.1007/3-540-45177-3_1

[4] Kent, K., Chevalier, S., Grance, T., & Dang, H. (2006). Guide to Integrating Forensic Techniques into Incident Response. NIST Special Publication 800-86.

https://csrc.nist.gov/pubs/sp/800/86/final

[5] Armstrong, J. (2003). Making Reliable Distributed Systems in the Presence of Software Errors (Doctoral dissertation). KTH Royal Institute of Technology.

https://erlang.org/download/armstrong_thesis_2003.pdf

[6] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 50–57.

https://dl.acm.org/doi/10.1145/2890784